

PROMPT-TRD: DECISION FUNNEL & ALGORITHM INDEX

0. SYSTEM PROMPT & BASE TEMPLATE

Інструкція для ІІІ (Промпт): «Використовуй наступний шаблон C++. Пиши **ТІЛЬКИ** валідний C++ код у функції main або як допоміжні функції. Умова: . Формат вхідних даних: . Формат вихідних даних: . Обмеження по часу: Обмеження по пам'яті. Це задача зі спортивного програмування, використай підхід який вкладається в обмеження по часу. Думай і відповідай виключно англійською мовою»

```
#include <bits/stdc++.h>
using namespace std;
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
int main() {
    cin.tie(0)->sync_with_stdio(0);
    cin.exceptions(cin.failbit);

    return 0;
}
```

КРОК 1. РАДАР ТЕМ (Симптоми в умові задачі → Розділ)

- Міста, дороги, лабіринти, мережі, пропускна здатність, залежності → Дивись [Розділ 2 \(ГРАФИ\)](#)
- Запити на відрізках (L..R), оновлення елементів, динамічні масиви → Дивись [Розділ 1 \(СТРУКТУРИ ДАНИХ\)](#)
- Максимізація прибутку, кількість способів, вибір з обмеженнями → Дивись [Розділ 3 \(ДИНАМІКА\)](#)
- Рядки, підрядки, паліндроми, словник, пошук збігів → Дивись [Розділ 4 \(РЯДКИ\)](#)
- Прості числа, модульна арифметика (10^9+7), матриці, поліноми → Дивись [Розділ 5 \(МАТЕМАТИКА\)](#)
- Математичне сподівання, ймовірність, перестановки, "скільки існує..." → Дивись [Розділ 6 \(КОМБІНАТОР\)](#)
- Координати, точки, площі, багатокутники, перетини → Дивись [Розділ 7 \(ГЕОМЕТРІЯ\)](#)
- "Знайти найменше можливе максимальне X", пошук оптимуму → Дивись [Розділ 8 \(ПОШУК / ОПТИМІЗАЦІЯ\)](#)
- Два гравці, оптимальна гра, виграш/програш, камінці → Дивись [Розділ 9 \(ТЕОРІЯ ІГОР\)](#)
- Жадібний вибір, розклад, розбиття задачі на менші підзадачі → Дивись [Розділ 10 \(ПАРАДИГМИ\)](#)

КРОК 2. КОНТРОЛЕР ОБМЕЖЕНЬ (Обмеження N → Вимога до ІІІ)

Знайдіть максимальне обмеження N в умові задачі і додайте відповідну вимогу до базового промπτу (Крок 0), щоб уникнути Time Limit (TL).

- $N \leq 20$ → Вимагайте: **"Target complexity: $O(2^N)$ or $O(N!)$ "**
- $N \leq 500$ → Вимагайте: **"Target complexity: $O(N^3)$ "**
- $N \leq 10^4$ → Вимагайте: **"Target complexity: $O(N^2)$ "**
- $N \leq 2 \cdot 10^5$ → Вимагайте: **"Target complexity: strictly $O(N \log N)$ "**
- $N \geq 10^9$ → Вимагайте: **"Target complexity: $O(\log N)$ or $O(1)$ "** (Або $O(\sqrt{N})$ якщо це задача на прості числа).

РОЗДІЛ 1. СТРУКТУРИ ДАНИХ (DATA STRUCTURES)

ГІД ПО ВИБОРУ (HUMAN LOGIC)

- «Потрібно знаходити суму/мінімум на відрізку, і елементи часто змінюються» → Для простих операцій (сума) ідеально підійде **Fenwick**. Для мінімуму/максимуму — **Segment Tree**.
- «Додайте X до всіх чисел від L до R » або «Замініть всі числа на відрізку» → Це 100% **Lazy Propagation**. Без «лінивого» оновлення ви отримаєте Time Limit.
- «Масив взагалі не змінюється, але запитів на мінімум мільйони» → Тільки **Sparse Table**. Вона дає відповідь за $O(1)$.
- «Треба міняти місцями шматки масиву, розвертати їх або циклічно зсувати» → Звичайні дерева безсилі. Потрібне **Treap** (Декартове дерево по неявному ключу).
- «Згадайте, яким був масив 5 запитів тому» або «Знайти K -те число на відрізку» → **Persistent Segment Tree**.
- «Знайти мінімум у вікні довжини K , що ковзає» → **Monotone Queue** (або черга на 2 стеках).
- «Задача про точки на площині: знайдіть точки в прямокутнику» → **KD-Tree**.

БАЗОВІ СТРУКТУРИ (L3 / STL MASTERY)

- **Stack, Queue, Deque** / **Стек, Черга, Двостороння черга**: Базові контейнери для DFS/BFS або імітації процесів.
Імплементація: Використовувати `std::vector` замість `std::stack` для швидкості, або `std::deque` для додавання/видалення з обох кінців.
- **Hash Tables** / **Хеш-таблиці** (`unordered_map`, `unordered_set`): Пошук за $O(1)$.
Імплементація (КРИТИЧНО): Уникайте стандартного `std::hash`, інакше зловмисні тести на Codeforces/ICPC викличуть колізії і TL ($O(N^2)$). Завжди просіть ІІІ додати кастомний безпечний хеш (наприклад, `splitmix64` з `chrono::steady_clock`).
- **Self-balancing trees** / **Збалансовані дерева (AVL, Red-Black)**: Підтримка відсортованої множини з унікальними елементами за $O(\log N)$.
Імплементація: Це стандартні `std::set` або `std::map`. Якщо треба знати індекс елемента, просіть ІІІ підключити `pbds` (Policy Based Data Structures — `tree_order_statistics_node_update`).

ОСНОВНИЙ СТЕК (CORE STRUCTURES)

- **Fenwick Tree (BIT)** / **Дерево Фенвіка**: Найшвидша структура для сум на префіксах.
Коли: $N = 10^6$, дуже жорсткі ліміти пам'яті/часу.
Імплементація: 1-індексація, переходить через `x & -x`. Код займає 5-6 рядків.
- **Segment Tree** / **Дерево відрізків**: Надійна «робоча конячка».
Коли: Точкові оновлення та будь-які асоціативні операції (Min, Max, GCD).
Імплементація: Ітеративна версія на масиві $2 \cdot N$ (де батько це $i/2$) працює вдвічі швидше за рекурсивну на вказівниках.
- **Lazy Segment Tree** / **Дерево з відкладеними оновленнями**: Для масових змін.
Коли: «Додайте V до всіх A_i на $[L, R]$ ».
Імплементація: Рекурсивна структура з масивами `tree` та `lazy`. Обов'язкова функція `push_down()` перед тим, як спускатися до дітей.
- **Sparse Table** / **Розріджена таблиця**: Найшвидший статичний RMQ.
Коли: Масив не змінюється, $N \leq 2 \cdot 10^5$, запитів $> 10^6$.
Імплементація: Препроцесинг за $O(N \log N)$ у матрицю `st[20][MAXN]`. Відповідь за $O(1)$ через `__lg()`.
- **DSU (Union-Find)** / **Система неперетинних множин**: Зв'язність та компоненти.
Коли: Алгоритм Крускала, перевірка циклів, динамічна зв'язність.
Імплементація: Обов'язкова оптимізація стиснення шляху (`p[i] = find(p[i])`) та об'єднання за рангом/розміром.

ДЕРЕВА ТА ШЛЯХИ (TREES & PATHS)

- **LCA (Binary Lifting) / Найменший спільний предок:**

Симптом: Знайти відстань між двома вузлами в дереві або їхнього найближчого спільного предка.

Імплементація: Масив $up[LOGN][MAXN]$ (2^k -jumps). Спочатку вирівняти глибини вершин, потім одночасно піднімати обидві.

- **HLD (Heavy-Light Decomposition):**

Симптом: «Змінити значення вершини/ребра в дереві і знайдіть суму/мінімум на шляху від U до V ».

Імплементація: Два DFS. Перший рахує розмір піддерев, другий будує "важкі" шляхи (ланцюги). Запити до ланцюгів обробляються через Segment Tree.

- **Centroid Decomposition / Центроїдна декомпозиція:**

Симптом: «Порахуйте кількість пар вершин у дереві, відстань між якими дорівнює K ».

Імплементація: Знайти центроїд, вирішити задачу для шляхів, що проходять через нього, видалити центроїд і запустити рекурсію для піддерев (максимальна глибина логіки $\leq \log N$).

- **Pull/Push-technique on trees:** Парадигма оновлення значень.

Імплементація: Pull — перерахувати значення поточного вузла на основі значень його дітей. Push — скинути відкладені теги або стан від поточного вузла до дітей (як у Lazy).

АРСЕНАЛ ОСТАННЬОЇ НАДІЇ (ADVANCED / LAST RESORT)

- **Implicit Treap / Декартове дерево по неявному ключу:**

Симптом: Складні маніпуляції. «Розверніть відрізок $[L, R]$ » або «Перемістіть відрізок $[L, R]$ на початок масиву».

Імплементація: Вузли містять неявний індекс (сума розмірів лівого піддерева). Обов'язкові операції `split` та `merge`. Для реверсу використовується lazy-тег обернення. Пріоритети — суворий рандом (mt19937).

- **Persistent Segment Tree / Персистентне дерево відрізків:**

Симптом: Офлайн-запити до історії або 2D-запити. «Знайдіть K -те найменше число на відрізку $[L, R]$ ».

Імплементація (Важливо): Заборонити III використовувати `new` або вказівники. Пам'ять виділяється заздалегідь: `int lc[MAX], rc[MAX], sum[MAX]`. Кожне оновлення створює новий корінь і дублює $\log N$ вузлів.

- **Monotone Queue (Sliding queue using 2 stacks) / Вікно на двох стеках:**

Симптом: Потрібна черга, що підтримує агрегацію (мінімум, максимум, НСД), але стандартна `deque` не підходить через специфіку операції.

Імплементація: Два стеки (S_{in} та S_{out}). Додаємо в S_{in} , видаляємо з S_{out} . Якщо S_{out} порожній, пересипаємо всі елементи з S_{in} до S_{out} , перераховуючи їхні агреговані значення.

- **Monotone Stack / Монотонний стек:**

Симптом: «Для кожного числа знайдіть найближче більше число праворуч» (Next Greater Element) або «Знайдіть найбільший прямокутник у гістограмі».

Імплементація: Зберігати в стеку індекси елементів. Якщо новий елемент порушує монотонність, виштовхувати зі стеку і фіксувати відповідь.

- **Splay Tree / Сплей-дерево:**

Симптом: Потрібна структура типу Treap, але запити мають властивість локальності (до одних і тих самих елементів звертаються часто).

Імплементація: Базується на операції `splay(v)`, яка піднімає вузол в корінь через серію локальних поворотів дерева (zig-zig, zig-zag).

- **KD-Tree / К-вимірне дерево:**

Симптом: «Дано N точок на площині (2D/3D). Знайдіть найближчу до заданої точки» або «Скільки точок потрапляє в заданий прямокутник».

Імплементація: Просторова база даних. На кожному рівні рекурсії площа ділиться навпіл перпендикулярно до однієї з осей (X , потім Y , потім знову X).

РОЗДІЛ 2. ТЕОРІЯ ГРАФІВ (GRAPH THEORY)

ГІД ПО ВИБОРУ (HUMAN LOGIC)

- «Як найшвидше дістатися з точки А в точку Б?»
→ Ваги додатні: **Dijkstra**. Є від'ємні ваги/цикли: **Bellman-Ford**. Ваги однакові (або це сітка/лабіринт): **BFS**. Треба відстані між UCSA парами: **Floyd-Warshall**.
- «З'єднати всі міста проводами так, щоб сумарна довжина була мінімальною» → **MST (Kruskal)**.
- «Труби, пропускна здатність, перевезення вантажів» → **Max Flow**. Якщо за перевезення кожної одиниці вантажу треба ще й платити — **Min-Cost Max-Flow**.
- «Розподілити робітників по робочих місцях / Хлопців по дівчатах» → **Bipartite Matching**.
- «Умова типу АБО: Якщо я візьму річ А, не можна брати Б. Треба задовольнити всі умови» → **2-SAT**.
- «Є залежності: задачу А треба виконати перед задачею Б» → **Topological Sort**.
- «Знайти дороги, при закритті яких місто відріже від країни» → **Bridges / Cut Vertices**.

ОБХІД ТА ЗВ'ЯЗНІСТЬ (TRAVERSAL & CONNECTIVITY)

- **Graph Representation / Зберігання графу:**
Імплементація: Для $N \leq 10^5$ використовувати списки суміжності `vector<int> adj[N]`. Матриця суміжності `adj[N][N]` — ТІЛЬКИ якщо $N \leq 1000$, інакше Memory Limit.
- **BFS / Хвильовий алгоритм у сітці (Grid BFS):** Найкоротший шлях на незважених графах або в 2D-лабіринтах.
Імплементація: Створити масиви зміщень `int dx[] = {0, 1, 0, -1}`. Завжди перевіряти межі сітки ($x \geq 0$ && $x < R$) перед додаванням у чергу.
- **Topological Sorting / Топологічне сортування:** Порядок виконання залежних задач.
Імплементація: Алгоритм Кана (Kahn) з чергою та підрахунком `in_degree[v]`, або через DFS з додаванням вершин у масив при виході (потім розвернути масив).
- **SCC (Strongly Connected Components) / Сильна зв'язність:** Знайти цикли в орієнтованому графі та стиснути їх у DAG (Конденсація графу).
Імплементація: Алгоритми Тар'яна (Tarjan) або Косараджу (Kosaraju). Конденсація: після SCC побудувати новий граф, де компоненти є вершинами.
- **Bridges & Cut Vertices / Мости та шарніри (Biconnected components):** Вразливі ребра/вузли.
Імплементація: Обхід DFS з використанням масивів `tin[v]` (час входу) та `low[v]` (мінімальний час досягної вершини). Ребро є мостом, якщо `low[to] > tin[v]`.
- **Normal trees / DFS trees:** Розбиття ребер загального графу на деревні (tree edges) та зворотні (back edges).
Імплементація: Якщо під час DFS зустріли відвідану вершину — це зворотне ребро, яке утворює цикл.

НАЙКОРОТШІ ШЛЯХИ ТА ЦИКЛИ (PATHS & CYCLES)

- **Dijkstra's Algorithm / Алгоритм Дейкстри:** Шляхи від однієї вершини (ваги ≥ 0).
Імплементація (КРИТИЧНО): Вимагати використання `std::priority_queue` з `greater`. Без цього ШІ може написати наївний $O(V^2)$ замість $O(E \log V)$.
- **Bellman-Ford / Алгоритм Форда-Беллмана (SPFA):** Від'ємні ваги та пошук від'ємних циклів.
Імплементація: Для $N = 10^5$ вимагати **SPFA** (Bellman-Ford з чергою). Цикл знайдено, якщо вершина потрапила в чергу більше V разів.
- **Floyd-Warshall / Алгоритм Флойда-Воршелла:** Усі найкоротші шляхи.
Імплементація: $O(V^3)$. Три вкладених цикли. Важливо: зовнішній цикл **завжди** по k (проміжна вершина).
- **Euler Cycles / Ейлерові цикли:** Шлях, що проходить по кожному ребру рівно один раз.
Імплементація: Алгоритм Хірхольцера (Hierholzer). Спочатку перевірити ступені вершин (всі парні для циклу). Використовувати стек.
- **Shortest Cycle / Найкоротший цикл у графі:**
Імплементація: Якщо незважений — запускати BFS з кожної вершини $O(V \cdot (V + E))$. Якщо зважений — алгоритм Дейкстри з кожної вершини, або Флойд-Воршелл.
- **K'th shortest path / К-й найкоротший шлях:**
Імплементація: Алгоритм Єна (Yen) або Дейкстра з чергою, що дозволяє відвідувати одну вершину K разів.

ОСТОВНІ ДЕРЕВА (MST) ТА ДЕРЕВА

- **Kruskal's Algorithm / Алгоритм Крускала:** Мінімальне кістякове дерево.
Імплементація: Відсортувати всі ребра за вагою, додавати в граф, уникаючи циклів за допомогою CHM (DSU). Складність $O(E \log E)$.
- **Prim's Algorithm / Алгоритм Прима:** Альтернатива для дуже щільних графів.
Імплементація: Аналог Дейкстри, але черга зберігає мінімальне ребро, що з'єднує додану частину з рештою графу.
- **Diameter and Centroid / Діаметр та Центр дерева:**
Імплементація: Діаметр = 2 запуски BFS (знайти найвіддаленішу вершину від випадкової, потім найвіддаленішу від неї).
- **Matrix Tree Theorem / Матрична теорема про дерева (Кірхгофа): [L4]**
Симптом: «Скільки існує різних остовних дерев у графі?»
Імплементація: Побудувати матрицю Лапласа (на діагоналі ступені вершин, на i, j місцях -1 , якщо є ребро). Відповідь — детермінант будь-якого мінору матриці розміром $(N - 1) \times (N - 1)$.

ПАРОСПОЛУЧЕННЯ ТА ПОКРИТТЯ (MATCHINGS & COVERS)

- **Bipartite matching / Паросполучення у двочастковому графі:** Максимальна кількість пар.
Імплементація: Алгоритм Куна (Kuhn) через DFS за $O(V \cdot E)$. Працює швидко на практиці для $N \leq 10^3$.
- **Hopcroft-Karp / Алгоритм Хопкрофта-Карпа:** Швидке паросполучення.
Коли: Жорсткий TL або великий граф ($N > 10^4$).
Імплементація: Комбінація BFS та DFS. Асимптотика $O(E\sqrt{V})$.
- **Min-cost matching / Паросполучення мінімальної ваги:** (Hungarian / Kuhn-Munkres).
Симптом: «Кожен робітник бере різну зарплату за різну роботу, знайдіть мінімальну/максимальну сумарну вартість».
Імплементація: Угорський алгоритм $O(V^3)$. Потребує побудови матриці вартостей.
- **König's theorem and Vertex Cover / Теорема Кеніга:** Взаємозв'язки у двочасткових графах.
Симптом: «Вибрати мінімальну кількість вершин, щоб кожне ребро торкалося хоча б однієї» (Min Vertex Cover).
Імплементація: У двочасткових графах: Vertex Cover = Max Matching. А найбільша незалежна множина (Max Independent Set) = V - Max Matching.
- **Hall's marriage theorem / Теорема Холла:**
Симптом: «Чи можливо ідеально з'єднати всіх з лівої частки з правою?»
Імплементація: Перевірка умови: для будь-якої підмножини X з лівої частки, кількість їхніх сусідів $\geq |X|$.
- **Maximal matching (general graphs) / Алгоритм Едмондса (Blossom): [L4]**
Симптом: Паросполучення у графі, де МОЖУТЬ бути цикли непарної довжини (НЕ двочастковий).
Імплементація: Алгоритм стиснення «квіток» (Blossom Algorithm). Дуже складний об'ємний код.

ПОТОКИ В МЕРЕЖАХ (FLOW NETWORKS)

[L4]

- **Edmonds-Karp / Dinic / Push-Relabel:** Максимальний потік (Max Flow).
Симптом: «Яку максимальну кількість води/даних можна передати від Джерела (S) до Стоку (T)».
Імплементація: Завжди вимагайте **Dinic** для звичайних задач (швидкий, працює за $O(V^2E)$) та **Push-Relabel** для дуже щільних графів або супер-великих обмежень. Забороніть Edmonds-Karp (він повільний).
- **Min-Cost Max-Flow (MCMF) / Мін-вартість Макс-потік:**
Симптом: Знайти потік заданої величини (або максимальний), мінімізувавши вартість проходження ребер.
Імплементація: Використовуйте Successive Shortest Path (Дейкстра з потенціалами або SPFA для пошуку збільшувальних шляхів у залишковій мережі).
- **Global Min-Cut / Глобальний мінімальний розріз:**
Симптом: Розділити граф на 2 компоненти, видаливши ребра мінімальної сумарної ваги (Джерела і Стоку НЕМАЄ).
Імплементація: Алгоритм Стоєра-Вагнера (Stoer-Wagner) за $O(V^3)$.
- **Graph Cut Inference / Project Selection Problem:**
Симптом: «Є проекти і ресурси. Проект приносить прибуток, ресурс коштує грошей. Проекти вимагають певних ресурсів. Максимізувати прибуток».
Імплементація: Зводиться до Max Flow / Min Cut. (Сток - прибуток, Джерело - втрати, ребра безкінечності для залежностей).

АРСЕНАЛ ОСТАННЬОЇ НАДІЇ ТА НІЩЕВІ (ADVANCED / LAST RESORT)

- **2-SAT (2-Satisfiability): [L4]**

Симптом: Є N змінних (True/False). Умови задаються як: «Якщо A , то не B », або «Хоча б одне з X або Y має бути правдою». Треба знайти, чи існує комбінація.

Імплементація: Побудувати граф імплікацій (умова $x \vee y$ перетворюється на ребра $\neg x y$ та $\neg y x$). Запустити алгоритм SCC. Якщо змінна X та $\neg X$ потрапили в одну компоненту сильної зв'язності — рішення не існує.

- **Dynamic graphs (extra book-keeping) / Динамічні графи: [L4]**

Симптом: До графу постійно додаються і ВИДАЛЯЮТЬСЯ ребра. Треба відповідати на запити про зв'язність між цими змінами.

Імплементація: Офлайн-рішення: Divide and Conquer по часу + CHM (DSU) з можливістю скасування операцій (Rollback DSU, без стиснення шляху). Складність $O(N \log^2 N)$.

- **Min. Path Cover / Покриття мінімальною кількістю шляхів: [L4]**

Симптом: «Яку мінімальну кількість неперетинних шляхів треба, щоб покрити всі вершини графу?» (Часто — маршрути таксі, літаків).

Імплементація: Для DAG (ациклічних орієнтованих графів) зводиться до Bipartite Matching. Відповідь = V - (Max Matching).

- **Graphical Sequences / Графічні послідовності: [L4]**

Симптом: «Дано масив чисел. Чи можливо побудувати граф, де ці числа будуть ступенями вершин?»

Імплементація: Алгоритм Хавела-Хакімі (Havel-Hakimi). Сортувати за спаданням, брати найбільше число K , віднімати 1 від наступних K чисел. Якщо з'явилося від'ємне число — неможливо.

- **Vertex coloring (3^n special case of set cover) / Розфарбування вершин: [L4]**

Симптом: Розфарбувати граф мінімальною кількістю кольорів так, щоб суміжні вершини мали різні кольори.

Імплементація: Якщо кольорів 2 — це перевірка на двочастковість (BFS). Для загальних графів (NP-hard) — динаміка по підмножинах (SOS DP) за $O(3^N)$ або оптимізований Backtracking.

- **Edge coloring (Trees) / Розфарбування ребер у дереві: [L4]**

Симптом: Пофарбувати ребра дерева так, щоб жодні два суміжних ребра не мали однакового кольору.

Імплементація: Кількість кольорів завжди дорівнює максимальному ступеню вершини Δ . Робиться жадібно через DFS.

- **Lovasz toggle: [L4]**

Симптом: Вузька теоретико-ігрова / графова концепція для пошуку розбиттів графу з певними обмеженнями на ступені (теорема Ловаса).

Імплементація: Локальний пошук (фліпання стану вершин), доки не буде досягнута локально стабільна конфігурація.

Порада для промптів (Графи): Якщо задача вимагає побудувати граф з "умови", обов'язково напишіть III: «Read the input carefully to build the correct graph. Handle 1-based indexing of nodes properly.» — це найчастіша причина помилок 'Segmentation Fault' у моделі.

РОЗДІЛ 3. ДИНАМІЧНЕ ПРОГРАМУВАННЯ (DP)

ГІД ПО ВИБОРУ (HUMAN LOGIC)

- «Вибрати предмети, щоб сума ваг $\leq W$, а вартість максимальна» $\rightarrow$ Knapsack (Рюкзак).
- «З двох рядків / масивів треба щось викреслити, щоб вони стали однаковими» $\rightarrow$ LCS (Найдовша спільна підпоследовність).
- «Знайти найдовший ланцюжок чисел, який зростає» $\rightarrow$ LIS. Якщо $N \leq 10^5$, обов'язково вимагати $O(N \log N)$ рішення.
- «Об'єднати сусідні елементи масиву, вартість об'єднання залежить від їхньої суми» $\rightarrow$ Це DP на відрізках (Interval DP). Класичні симптоми: злиття камінців, множення матриць.
- «Є N міст ($N \leq 20$), треба відвідати всі (TSP)» або «Розбити на пари» $\rightarrow$ DP по масках (Bitmask DP).
- «Матриця/Сітка з дуже вузькою шириною ($M \leq 12$), треба замостити її фігурками (доміно)» $\rightarrow$ Broken Profile DP (Динаміка по профілю).
- «Моя $O(N^2)$ динаміка ловить TL. Формула виглядає як $dp[i] = \min(dp[j] + X_i \cdot Y_j)$ » $\rightarrow$ Вимагайте оптимізацію Convex Hull Trick (СНТ).

БАЗОВЕ ДИНАМІЧНЕ ПРОГРАМУВАННЯ (L3)

- Knapsack / Задача про рюкзак (0/1 та Unbounded):
Симптом: Обмелена місткість, максимізація прибутку.
Імплементація: Для економії пам'яті (щоб уникнути ML) вимагати 1D-масив: $dp[w]$. Для 0/1 рюкзак внутрішній цикл йде справа наліво (від W до w_i). Для нескінченного (Coin Change) — зліва направо.
- Coin Change / Розбиття чисел:
Симптом: Скількома способами можна набрати суму S монетами / Яка мінімальна кількість монет?
Імплементація: Динаміка $dp[S]$. Якщо шукаємо мінімум — ініціалізувати масив INF (наприклад, 10^9).
- LCS (Longest Common Subsequence) / Найдовша спільна підпоследовність:
Симптом: Знайти спільний порядок елементів у двох масивах/рядках.
Імплементація: 2D масив $dp[i][j]$. Відновлення відповіді (backtracking) робиться з кінця масиву dp до $dp[0][0]$.
- LIS (Longest Increasing Subsequence) / Найдовша зростаюча підпоследовність:
Імплементація (КРИТИЧНО): Якщо $N = 10^5$, $O(N^2)$ динаміка впаде. Вимагати III писати за $O(N \log N)$ з використанням масиву `tails` (або `dp`) та функції `std::lower_bound`.
- Memoization (Top-down DP) / Лінива динаміка:
Симптом: Складна рекурсивна логіка або ігри (Game Theory), де важко придумати порядок циклів.
Імплементація: Масив мемо завжди ініціалізувати -1 (через `memset(мемо, -1, sizeof(мемо))`). На початку рекурсії перевіряти: `if (мемо[state] != -1) return мемо[state];`.
- Linear DP / Лінійні задачі:
Симптом: Стрибки коника, жабки, кількості шляхів у сітці. Стан залежить тільки від 1-2 попередніх кроків.
Імплементація: Часто можна оптимізувати пам'ять до $O(1)$, зберігаючи тільки $dp[i-1]$ та $dp[i-2]$.

ПРОСУНУТЕ DP ТА ДИНАМІКА НА СТРУКТУРАХ (L4)

- **Dynprog over intervals / Динаміка на відрізках (Interval DP):**

Симптом: Злиття сусідніх елементів у масиві, множення ланцюга матриць, паліндроми в рядку.

Імплементація (КРИТИЧНО): III часто плутає порядок циклів! Зовнішній цикл **обов'язково** має бути по довжині відрізка (len від 2 до N), внутрішній по початку (i), а всередині цикл перебору точки розбиття (k від i до $j - 1$).

- **Dynprog over subsets / DP по масках (Bitmask DP):**

Симптом: $N \leq 20$. Задачі про комівояжера (TSP), гамільтонові шляхи, оптимальні розбиття на пари.

Імплементація: Стан `dp[mask][last_node]`. Для перевірки біта використовувати `(mask > i) & 1`.

- **3^n Set Cover / Перебір підмасок:**

Симптом: Треба розбити $N \leq 15$ елементів на оптимальні групи (неперетинні підмножини).

Імплементація: Подвійний цикл. Зовнішній по `mask`, внутрішній по підмасках: `for (int sub = mask; sub > 0; sub = (sub - 1) & mask)`. Це працює строго за $O(3^N)$, а не $O(4^N)$.

- **Dynprog over trees / Динаміка на деревах (Tree DP):**

Симптом: Максимальна незалежна множина в дереві, мінімальне вершинне покриття.

Імплементація: Зазвичай DFS, який повертає два стани для кожної вершини U : `dp[u][0]` (не беремо вершину) і `dp[u][1]` (беремо вершину). Обчислюється знизу вгору (bottom-up).

- **Broken Profile DP / Динаміка по профілю:**

Симптом: Замощення сітки $N \times M$ доміношками, де $\min(N, M) \leq 12$.

Імплементація: Стан визначається як `dp[cell_index][mask]`, де `mask` — це наявність фігурок у попередньому рядку / зсунутому профілі.

- **Number of paths / Shortest path in a DAG (DP на орієнтованих ациклічних графах):**

Симптом: Кількість способів дійти від старту до фінішу в графі, де немає циклів.

Імплементація: Спочатку зробити **Topological Sort**. Потім просто ітеруватися по вершинах у топологічному порядку: `dp[to] = (dp[to] + dp[u]) % MOD`.

- **Dynprog over probabilities / Динаміка за ймовірностями:**

Симптом: «Знайти математичне сподівання кількості кроків до фінішу» або «Ймовірність виграти».

Імплементація: Математичне сподівання (Expected Value) майже завжди обчислюється з **кінця до початку** ($E[finish] = 0$, $E[u] = 1 + \sum (P(v) \cdot E[v])$). Ймовірності (Probabilities) обчислюються з **початку до кінця**.

СУПЕР-НІШЕВІ ПАТЕРНИ (L4)

- **Bitonic cycle (TSP):**

Симптом: Знайти шлях комівояжера, який строго йде зліва направо, а потім справа наліво.

Імплементація: Впорядкувати точки за X . Стан `dp[i][j]` — це кінець першого шляху і кінець другого шляху.

- **Log partitioning / Логарифмічне розбиття:**

Симптом: Динаміка, де один із параметрів дуже великий, але його можна зменшувати лише діленням на 2.

Імплементація: Розмір масиву для цього параметра буде всього близько $\approx \log_2(\text{MAX_VAL})$ (наприклад, 30 або 60).

Цей блок потрібен, коли ви правильно склали рівняння DP ($O(N^2)$ або $O(N^3)$), але воно дає Time Limit. Ці алгоритми знижують складність на порядок (наприклад, з $O(N^2)$ до $O(N \log N)$).

- **Convex Hull Trick (CHT) / Оптимізація опуклої оболонки:**

Симптом: Перехід виглядає як лінійна функція $dp[i] = \min_j (dp[j] + A_i \cdot B_j)$. Тобто є добуток параметрів від i та j .
Імплементація: Якщо B_j відсортовані (монотонні), використовуємо двосторонню чергу (`std::deque`), що зберігає прямі $y = kx + b$. Якщо не відсортовані — III має написати **Li Chao Tree** або динамічний CHT (через `std::set` з перевантаженим оператором). Складність падає з $O(N^2)$ до $O(N)$ або $O(N \log N)$.

- **Divide and Conquer DP / Розділай і володарюй в DP:**

Симптом: DP розбиття на масиви $dp[k][i] = \min_{j < i} (dp[k-1][j] + cost(j, i))$. Точка оптимуму $opt[k][i]$ є монотонною: $opt[k][i] \leq opt[k][i+1]$.

Імплементація: Оптимізуємо зовнішній рівень k . Рекурсивна функція `solve(l, r, opt_l, opt_r)`. Знаходимо оптимум для $mid = (l + r)/2$, а потім запускаємо рекурсію `solve(l, mid-1, opt_l, opt_mid)` та `solve(mid+1, r, opt_mid, opt_r)`. Складність падає з $O(K \cdot N^2)$ до $O(K \cdot N \log N)$.

- **Knuth Optimization / Оптимізація Кнута:**

Симптом: Динаміка на відрізках (Interval DP) виду $dp[i][j] = \min(dp[i][k] + dp[k][j]) + cost(i, j)$.

Імплементація: Точка розбиття $opt[i][j]$ затиснута в межах $opt[i][j-1] \leq opt[i][j] \leq opt[i+1][j]$. Внутрішній цикл перебирає k тільки в цих межах, а не від i до j . Складність падає з $O(N^3)$ до $O(N^2)$.

- **RMQ (Sparse Table / Segment Tree) для переходів DP:**

Симптом: Перехід виду $dp[i] = \min(dp[j]) + C$ для всіх j у якомусь «вікні» або відрізьку $[L_i, R_i]$.

Імплементація: Зберігати значення попереднього шару DP у Дереві Відрізків або Розрідженій таблиці і робити запит мінімуму за $O(\log N)$ замість циклу.

- **2^k -jumps / Бінарні підйоми у DP:**

Симптом: «Скільки кроків треба, щоб дістатися кудись, якщо ми знаємо наступний оптимальний крок», або сума ваг на довгому шляху в дереві/графі функцій (functional graph).

Імплементація: Попередньо обчислити таблицю `jump[LOG][N]` — куди ми потрапимо, зробивши 2^k кроків зі стану i . Дозволяє відповідати на запити стрибків за $O(\log K)$ замість $O(K)$.

Порада для промптів (Динаміка): Якщо відповідь може бути великою, обов'язково нагадайте III: «Use long long for the DP table and return value. Apply modulo 1000000007 (or 998244353) at EVERY addition/subtraction if required.» Це врятує вас від 80% помилок Wrong Answer.

РОЗДІЛ 4. ОБРОБКА РЯДКІВ (STRINGS)

ГІД ПО ВИБОРУ (HUMAN LOGIC)

- «Знайти всі входження одного короткого слова (шаблону) у довгий текст» → KMP або Z-функція.
- «Є великий словник (багато слів). Знайти, які з них є в тексті» → Це 100% Aho-Corasick.
- «Швидко порівнювати два довгих підрядки на рівність» → Rolling Polynomial Hashing.
- «Скільки унікальних підрядків у тексті?» або «Знайти найдовший спільний підрядок» → Suffix Array + LCP (це безпечніша альтернатива Суфіксному дереву).
- «Знайти найдовший паліндром у тексті» → Manacher's Algorithm.
- «Шукати слова за їхнім префіксом (автозаповнення)» або «Максимальний XOR двох чисел» → Trie (Префіксне дерево).

ОСНОВНИЙ СТЕК (CORE STRUCTURES)

- **Z-function / Z-функція:**
Симптом: Знайти довжину найбільшого спільного префікса між самим рядком і всіма його суфіксами.
Імплементація: Масив `z[i]`. Код у 10 рядків (через 'L' та 'R' межі знайденого співпадіння). Для пошуку підрядка шукають Z-функцію від рядка `pattern + "#" + text`.
- **KMP (Knuth-Morris-Pratt) / Префікс-функція:**
Симптом: Знайти найменший період рядка або всі входження підрядка.
Імплементація: Масив `pi[i]` (довжина найдовшого власного префікса, що збігається з суфіксом). Пошук аналогічний: `pattern + "#" + text`.
- **Polynomial Hashing & Rabin-Karp / Поліноміальне хешування:**
Симптом: Порівняння підрядків за $O(1)$, пошук циклів.
Імплементація (КРИТИЧНО!): На сучасних контекстах 100% є анти-хеш тести, що ламають одномодульний хеш (або `unsigned long long`). Завжди вимагайте від III: "Use Double Polynomial Hashing with two large prime moduli (e.g., $10^9 + 7$ and $10^9 + 9$) and randomized base $B > 256$."
- **Tries (Prefix Tree) / Префіксне дерево:**
Симптом: Зберігання словника, рух по літерах, побітовий пошук (наприклад, "знайти X у масиві, щоб $X \oplus Y$ було максимальним").
Імплементація: Двовимірний масив `trie[MAX_NODES][26]` або вектор структур. Для побітового XOR — розмір алфавіту 2 (0 та 1).
- **Naive & Letter position lists:**
Симптом: Умова з малими обмеженнями або треба швидко дізнатися, де зустрічається кожна літера.
Імплементація: `vector<int> pos[26]`. Знайти перше входження літери після індексу i можна через `std::lower_bound`.

АРСЕНАЛ ОСТАННЬОЇ НАДІЇ ТА НІЩЕВІ (L4 BOSS)

- **Suffix Array + LCP / Суфіксний масив: [L4]**
Симптом: Потужний аналіз підрядків. Лексикографічне сортування всіх суфіксів. Задачі типу «найдовший підрядок, що зустрічається хоча б K разів».
Імплементація: Вимагайте від III "Construct Suffix Array in $O(N \log N)$ and build the LCP (Longest Common Prefix) array in $O(N)$ using Kasai's algorithm." Ніколи не використовуйте $O(N^2 \log N)$ сортування.
- **Suffix Tree / Суфіксне дерево: [L4]**
Симптом: Візуальне представлення всіх підрядків.
Імплементація: Алгоритм Укконена (Ukkonen's) за $O(N)$. **Увага:** Код екстремально складний і схильний до багів. У 99% випадків Суфіксне дерево можна (і треба) замінити на **Suffix Array + LCP**.
- **Aho-Corasick Automaton / Автомат Ахо-Корасік: [L4]**
Симптом: Дано словник з K слів (сумарна довжина M) і текст довжини N . Знайти всі слова в тексті за $O(N + M)$.
Імплементація: Префіксне дерево (Trie) + суфіксні посилання (suffix links), які будуються через BFS. Для оптимізації переходів часто додають **terminal link** (шлях до найближчого термінального стану).
- **Manacher's Algorithm / Алгоритм Манакера: [L4]**
Симптом: Точна фраза: «Знайти найдовший паліндром як підрядок» або «Скільки паліндромних підрядків у тексті».
Імплементація: Рахує масив `d[i]` — радіус паліндрома з центром у i за строги $O(N)$. Для пошуку парних паліндромів між символами вставляють фіктивні (наприклад, `#`).

- **Palindrome subsequences / Паліндромні підпоследовності:**

Симптом: Не суцільний підрядок, а можна "викреслювати" букви.

Імплементація: Це вже Динамічне Програмування (Interval DP). Стан: $dp[l][r]$. Якщо $S[l] == S[r]$, то $dp = dp[l+1][r-1] + 2$.

- **Longest Common Substring / Найдовший спільний підрядок (LCS-string):**

Симптом: Спільна частина для 2-х або більше рядків (саме суцільна).

Імплементація: Для двох рядків — бінарний пошук по довжині + подвійний хеш. Для кількох рядків — склеїти їх через спецсимволи (A # B \$ C %) і побудувати **Suffix Array** + **LCP**.

Порада для промптів (Рядки): Завжди уточнюйте у промпті обмеження алфавіту: **"The string contains only lowercase English letters [a-z]"**. Це дозволить ШІ оптимізувати пам'ять (масиви розміром 26 замість 256 чи map).

РОЗДІЛ 5. МАТЕМАТИКА ТА ТЕОРІЯ ЧИСЕЛ (MATH)

ГІД ПО ВИБОРУ (HUMAN LOGIC)

- «Порахувати відповідь по модулю $10^9 + 7$ і є операція ділення» → Ділити по модулю не можна! Потрібен **Modular Inverse** (Обернене за модулем).
- «Знайти всі прості числа до 10^7 » → **Sieve of Eratosthenes** (Решето Ератосфена).
- «Знайти $A^B \pmod{M}$, де $B = 10^{18}$ » → **Binary Exponentiation** (Швидке піднесення до степеня).
- «Знайти N -те число Фібоначчі або розв'язати рекуренту для $N = 10^9$ » → **Matrix Exponentiation** (Матричне піднесення до степеня).
- «Перемножити два дуже великі числа або два поліноми» → **Fast Fourier Transform (FFT)**.
- «Чи є число 10^{18} простим? / Розкласти 10^{18} на множники» → **Miller-Rabin** (тест на простоту) та **Pollard-Rho** (факторизація).
- «Розв'язати систему лінійних рівнянь або знайти визначник матриці» → **Gaussian Elimination**.
- «Максимізувати прибуток, маючи купу обмежень-нерівностей виду $ax + by \leq C$ » → **Simplex Algorithm**.

БАЗОВА АЛГЕБРА ТА АРИФМЕТИКА (L3)

- **Binary Exponentiation** / Швидке піднесення до степеня:
Симптом: Обчислення $A^B \pmod{M}$ за $O(\log B)$.
Імплементація: Ітеративний алгоритм (цикл `while (B > 0)`). Вимагайте від ПШ завжди використовувати `long long` для уникнення переповнення при множенні.
- **Sieve of Eratosthenes** / Решето Ератосфена:
Симптом: Факторизація багатьох чисел або масив простих чисел до 10^7 .
Імплементація: Модифікація з масивом `min_prime[i]` (SPF - Smallest Prime Factor) дозволяє розкладати будь-яке число X на множники за $O(\log X)$.
- **Modular Multiplicative Inverse** / Обернене за модулем:
Симптом: Треба обчислити $(A/B) \pmod{M}$. Це дорівнює $(A \cdot B^{-1}) \pmod{M}$.
Імплементація: Якщо M — просте число, використовуємо **Fermat's Little Theorem** (Мала теорема Ферма): $B^{-1} \equiv B^{M-2} \pmod{M}$. Якщо M складене, але взаємно просте з B , використовуємо **Extended Euclidean Algorithm**.
- **Extended Euclidean Algorithm** / Розширений алгоритм Евкліда:
Симптом: Знайти коефіцієнти x та y у Діофантовому рівнянні: $A \cdot x + B \cdot y = \gcd(A, B)$.
Імплементація: Рекурсивна функція `extGCD(a, b, x, y)`. Відповідь для x може бути від'ємною, її треба правильно зсунути по модулю.
- **Euler's Phi function** / Функція Ейлера (φ):
Симптом: Знайти кількість чисел від 1 до N , які взаємно прості з N .
Імплементація: Якщо треба знайти для одного числа — обчислюється через його факторизацію за $O(\sqrt{N})$. Якщо для всіх чисел до N — обчислюється через модифіковане Решето Ератосфена за $O(N \log \log N)$.
- **Numeric Root-finding** / Чисельний пошук коренів:
Симптом: Знайти X , при якому монотонна функція дорівнює Y (Binary Search), або знайти екстремум (мінімум/максимум) опуклої функції (Ternary Search).
Імплементація: Для дробних чисел використовуйте фіксовану кількість ітерацій (наприклад, `for (int i = 0; i < 100; ++i)`) замість перевірки `R - L > EPS`, щоб уникнути нескінченних циклів через втрату точності `double`.

ЛІНІЙНА АЛГЕБРА ТА МАТРИЦІ (L3 / L4)

- **Matrix Exponentiation** / Матричне піднесення до степеня:
Симптом: Обчислити N -й член лінійної рекуренту (як Фібоначчі) для $N \geq 10^{18}$.
Імплементація: Побудувати матрицю переходів розміру $K \times K$ і піднести її до степеня N за $O(K^3 \log N)$. Всі множення робити по модулю.
- **Gaussian Elimination** / Метод Гаусса:
Симптом: Розв'язати систему рівнянь, знайти ранг матриці або визначник (Determinant).
Імплементація (КРИТИЧНО): Якщо операції йдуть у полі по модулю 2 (рівняння з XOR) — обов'язково використовувати `std::bitset` для прискорення в 64 рази ($O(N^3/64)$).

ПРОСУНУТА ТЕОРІЯ ЧИСЕЛ (L4 BOSS)

- **Miller-Rabin Primality Test / Тест Міллера-Рабіна:**

Симптом: Перевірити, чи є велике число $N \leq 10^{18}$ простим за $O(\log N)$.

Імплементація: Детермінована версія для 64-бітних чисел використовує перевірку по базах: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 37, 41, 43, 47, 53, 59, 67, 71, 73, 79, 83, 89, 97. Обов'язково вимагати у ШІ множення через `__int128`, щоб уникнути переповнення під час $(A * B) \% M$.

- **Pollard-Rho Algorithm / Алгоритм Полларда-Ро:**

Симптом: Швидко розкласти дуже велике число ($N \leq 10^{18}$) на прості множники.

Імплементація: Алгоритм пошуку циклу (через черепаху та зайця). Працює в парі з Міллером-Рабіном за $O(N^{1/4})$.

- **Chinese Remainder Theorem (CRT) / Китайська теорема про остачі:**

Симптом: Знайти найменше X , яке дає задані остачі при діленні на задані модулі: $X \equiv a_1 \pmod{m_1}, \dots, X \equiv a_k \pmod{m_k}$.

Імплементація: Якщо модулі не є взаємно простими, використовуйте розширений алгоритм Евкліда для їх покрокового злиття. Обережно з переповненнями!

- **Fast Fourier Transform (FFT / NTT) / Швидке перетворення Фур'є:**

Симптом: Потрібно перемножити два довгих масиви/поліноми так, щоб $C_k = \sum A_i \cdot B_{k-i}$, за $O(N \log N)$.

Імплементація (СУПЕР КРИТИЧНО!): 1) Вимагати від ШІ **iterative in-place FFT** (рекурсивна впаде по TL). 2) Якщо задача вимагає точних цілих чисел по модулю 998244353 — вимагати **NTT (Number Theoretic Transform)**, він працює без втрати точності `double`.

- **Simplex Algorithm / Симплекс-метод:**

Симптом: Задачі лінійного програмування (Linear Programming). Знайти максимум функції $C^T X$ за умов $AX \leq B$ та $X \geq 0$.

Імплементація: Надзвичайно складний код. Вимагайте у ШІ **Two-Phase Simplex Algorithm** для обробки від'ємних значень у векторі B .

АРСЕНАЛ ОСТАННЬОЇ НАДІЇ ТА НІШЕВІ КОНЦЕПЦІЇ (LAST RESORT)

- **Golden Section Search / Пошук золотим перетином:**

Симптом: Пошук екстремуму функції, яку не можна диференціювати, альтернатива Ternary Search. Вимагає менше обчислень функції.

- **Newton's Method / Метод Ньютона (Дотичних):**

Симптом: Швидкий пошук кореня рівняння $f(x) = 0$, якщо можна легко обчислити похідну $f'(x)$. Формула: $x_{n+1} = x_n - f(x_n)/f'(x_n)$.

- **Numeric Integration / Чисельне інтегрування:**

Симптом: Знайти площу під кривою, заданою складною формулою.

Імплементація: Формула Сімпсона (Simpson's rule) дає найкращу точність на олімпіадах.

- **Vieta Root Jumping / Стрибки Вієта:**

Симптом: Складна задача на теорію чисел. Задано відношення, наприклад $(a^2 + b^2)/(ab + 1) = k$. Треба довести щось або знайти інші пари.

Імплементація: Математичний трюк спуску. Розглядається як квадратне рівняння відносно однієї змінної, з якого через теорему Вієта генерується менший розв'язок.

- **Hensel Lifting / Лема Гензеля:**

Симптом: Знайти корінь многочлена по модулю P^K (ступінь простого числа).

Імплементація: Спочатку знаходиться корінь по модулю P , а потім ітеративно піднімається до $P^2, P^3 \dots P^K$ (схоже на метод Ньютона).

- **Quadratic Reciprocity / Квадратичний закон взаємності:**

Симптом: Визначити, чи існує таке x , що $x^2 \equiv a \pmod{P}$ (Символ Лежандра).

- **Frobenius Number / Число Фробеніуса:**

Симптом: Дано монети номіналом A і B ($\gcd(A, B) = 1$). Яку максимальну суму НЕ МОЖНА набрати? Відповідь: $A \cdot B - A - B$.

Порада для промптів (Математика): Для математичних задач ШІ дуже часто забуває зсувати від'ємні числа по модулю. Завжди додавайте інструкцію: **"Ensure modulo operations handle negative numbers correctly: $(X \% M + M) \% M$ ".**

РОЗДІЛ 6. КОМБІНАТОРИКА ТА ЙМОВІРНІСТЬ (COMBINATORICS)

ГІД ПО ВИБОРУ (HUMAN LOGIC)

- «Скількима способами вибрати K предметів з N ? ($N \leq 10^5$)» → Binomial Coefficients (nCr).
- «Кількість способів, щоб жодна з умов НЕ виконалася» → Inclusion-Exclusion Principle (PIE).
- «Правильні дужкові послідовності» або «Розрізати багатокутник на трикутники» → Catalan Numbers.
- «Розкласти однакові яблука по різних кошиках ($x_1 + x_2 + \dots = N$)» → Stars and Bars.
- «Багатокутник на цілих координатах. Точки всередині / Площа» → Pick's Theorem (Теорема Піка).
- «Скільки унікальних розфарбувань, якщо повороти вважаються однаковими?» → Burnside's Lemma.

ОСНОВНИЙ СТЕК (CORE MATH)

- Computation of binomial coefficients / Біноміальні коефіцієнти (nCr):
Симптом: Обчислити $C_n^k \pmod{M}$.
Імплементація (КРИТИЧНО!): Вимагайте: "Precompute factorials and their modular inverses in $O(N)$ using Fermat's Little Theorem. Answer nCr in $O(1)$." Для $N \leq 1000$ можна використовувати трикутник Паскаля.
- Inclusion/Exclusion (PIE) / Принцип включення-виключення:
Симптом: Перетин множин. «Знайти кількість чисел до N , які не діляться на жодне з масиву A ».
Імплементація: Бітові маски (від 1 до $2^M - 1$). Непарна кількість виставлених бітів — додаємо, парна — віднімаємо.
- Catalan number / Числа Каталана:
Симптом: Неперетинні хорди, бінарні дерева. Перші числа: 1, 1, 2, 5, 14, 42.
Імплементація: Формула: $C_n = \frac{1}{n+1} 2^{nn}$. Обчислювати ділення на $n+1$ через Modular Inverse!
- Stars and Bars / Кулі та перегородки:
Симптом: Цілочисельні розв'язки рівняння $x_1 + x_2 + \dots + x_k = N$ ($x_i \geq 0$).
Імплементація: Відповідь дорівнює $N + K - 1 \choose K - 1$. Зводиться до nCr запиту.
- Pick's theorem / Теорема Піка:
Симптом: Геометрія на сітці. Площа чи цілі точки.
Імплементація: $Area = I + B/2 - 1$ (I — точки всередині, B — точки на межі через $\gcd(dx, dy)$).

ТЕОРІЯ ЙМОВІРНОСТЕЙ ТА ADVANCED (L4 BOSS)

- Linearity of Expectation / Лінійність математичного сподівання:
Симптом: «Знайти очікувану кількість подій X ».
Імплементація: $E[A + B] = E[A] + E[B]$, навіть якщо події залежні. Розбийте складну подію на суму простих індикаторів (1 або 0) і знайдіть ймовірність кожного.
- Pigeon-hole principle / Принцип Діріхле:
Симптом: «Доведіть, що існує...» або підмасив з сумою, кратною N .
Імплементація: Префіксних сум $N + 1$, а остач N . Дві суми матимуть однакову остачу, їх різниця ділиться на N .
- Lucas' Theorem / Теорема Люка: [L4]
Симптом: $nCr \pmod{P}$, де N, R величезні (10^{18}), а P — мале просте число ($P \leq 10^5$).
Імплементація: N та R переводяться в систему числення з основою P , біноміальний коефіцієнт = добуток n_iCr_i по кожній "цифрі".
- Burnside's Lemma & Pólya / Лема Бернсайда: [L4]
Симптом: Підрахунок кількості об'єктів з точністю до симетрії.
Імплементація: $Ans = \frac{1}{|G|} \sum |X^g|$. Сума кількості розфарбувань, нерухомих для кожної симетрії, поділена на кількість симетрій.

Порада для промптів (Ймовірність): Якщо задача вимагає вивести ймовірність у вигляді нескоротного дроби $P/Q \pmod{M}$, завжди додавайте в промпт: "Output the probability as an integer $P \cdot Q^{-1} \pmod{M}$ using Fermat's Little Theorem for modular inverse."

РОЗДІЛ 7. ГЕОМЕТРІЯ ТА СКАНУВАННЯ (GEOMETRY & SWEEPING)

ГІД ПО ВИБОРУ (HUMAN LOGIC)

- «Знайти площу дивного багатокутника» або «Чи перетинаються два відрізки?» → Cross Product (Векторний добуток). Ніяких рівнянь прямих $y = kx + b$, тільки вектори!
- «Натягнути гумку навколо всіх точок» → Convex Hull (Опукла оболонка).
- «Координати до 10^9 , але нам важливий лише їхній відносний порядок» → Coordinate Compression (Стиснення координат). Дозволяє перенести задачу в масив розміру N .
- «Дано 10^5 відрізків. Чи є хоч один перетин? / Знайти найближчі точки» → $O(N^2)$ впаде. Потрібен Line Sweeping (Скануюча пряма).
- «Точки навколо одного центру, треба знайти найкращий кут» → Angle Sweeping (Кутове сканування).
- «Відрізати частину багатокутника лінією і знайти площу залишку» → Polygon Cut.

БАЗОВА ГЕОМЕТРІЯ (L3: ВЕКТОРИ ТА БАГАТОКУТНИКИ)

- Coordinates, Vectors, Dot & Cross Product / Векторний та скалярний добуток:
Симптом: Будь-яка геометрія. Кути, відстані, перевірка поворотів.
Імплементація (КРИТИЧНО!): Векторний добуток 2D: $A_x B_y - A_y B_x$. Якщо він > 0 — поворот ліворуч, < 0 — праворуч, $= 0$ — точки на одній прямій. Вимагайте від ІІІ використовувати цілі числа (long long), а не double, щоб уникнути втрати точності!
- Oriented Area of Polygon / Орієнтована площа (Shoelace Formula):
Симптом: Площа багатокутника без самоперетинів.
Імплементація: Сума векторних добутків сусідніх вершин: $\frac{1}{2} |\sum (X_i \cdot Y_{i+1} - X_{i+1} \cdot Y_i)|$.
- Point in Polygon / Належність точки багатокутнику:
Симптом: Чи лежить точка всередині фігури.
Імплементація: Метод променя (Ray Casting). Випускаємо горизонтальний промінь від точки і рахуємо кількість перетинів з ребрами багатокутника. Непарна кількість — всередині, парна — зовні.
- Convex Hull / Опукла оболонка (Monotone Chain / Graham Scan):
Симптом: Знайти мінімальний опуклий багатокутник, що містить усі точки.
Імплементація: Алгоритм Monotone Chain (Ендрю) кращий за Грехема. Сортуюмо точки за X , потім за Y . Будуємо верхню і нижню оболонки через стек, перевіряючи повороти векторним добутком ($O(N \log N)$).

СИСТЕМИ СКАНУВАННЯ ТА ДИСКРЕТИЗАЦІЯ (SWEEPING)

- Coordinate-compression / Стиснення координат:
Симптом: Точки розкидані від -10^9 до 10^9 , але їх всього 10^5 . Треба використати Дерево Відрізків (яке не вмістить 10^9 елементів).
Імплементація: Зібрати всі X у вектор, відсортувати, видалити дублікати (`std::unique`). Замінити кожен X_i на його індекс через `std::lower_bound`.
- Line Sweeping / Скануюча пряма (Базова):
Симптом: Об'єднання площ прямокутників, максимальна кількість точок, що перекриваються.
Імплементація: Уявити вертикальну пряму, що їде зліва направо. "Події" — це початок відрізка/прямокутника (додати в структуру) і кінець (видалити). Події сортуються за X . Активні елементи лежать у `std::set` або Segment Tree (сортовані за Y).
- Angle Sweeping / Кутове сканування:
Симптом: Обертаємо промінь навколо точки. Знайти кут, що захоплює найбільше точок.
Імплементація (КРИТИЧНО!): ІІІ часто сортує точки через `atan2()`, що дає похибки double. Вимагайте: "Sort points radially using cross product and half-plane classification ($y > 0 \parallel (y == 0 \ \&\& \ x > 0)$) to maintain strict integer precision."

ПРОСУНУТЕ СКАНУВАННЯ ТА ГЕОМЕТРІЯ (L4 BOSS)

- **All segment-segment intersection / Перетин N відрізків: [L4]**

Симптом: Дано 10^5 відрізків. Чи є хоча б один перетин?

Імплементація: Sweepline. Події: початок відрізка і кінець відрізка. Активні відрізки зберігаються в `std::set`, відсортовані за Y -координатою. При додаванні відрізка перевіряємо перетин з його сусідами (вище і нижче) в `set`. При видаленні — перевіряємо перетин його сусідів між собою.

- **Closest pair of points / Найближча пара точок: [L4]**

Симптом: Знайти дві точки з мінімальною відстанню серед 10^5 точок.

Імплементація: Divide and Conquer ($O(N \log N)$) часто пишеться легше, ніж Sweepline. Розбиваємо площину вертикальною лінією, шукаємо мінімум d у лівій і правій половині. Потім перевіряємо точки у смужі $\pm d$ від лінії розбиття (кожна точка порівнюється не більше ніж з 6 іншими).

- **Polygon cut / Відрізання багатокутника прямим різом: [L4]**

Симптом: Багатокутник перерізає лінією $A \rightarrow B$. Знайти площу або нові вершини лівої частини.

Імплементація: Проходимо по ребрах. Якщо вершина лежить ліворуч від $A \rightarrow B$ (через Cross Product) — додаємо її. Якщо ребро перетинає $A \rightarrow B$ — знаходимо точку перетину і додаємо її.

- **Discrete second derivatives / Дискретні другі похідні (Difference Arrays): [L4]**

Симптом: Додавання лінійної прогресії ($V, V + D, V + 2D \dots$) на масив або параболи на відрізок $[L, R]$.

Імплементація: Аналог префіксних сум, але застосований двічі. Щоб додати арифметичну прогресію, робимо точкові зміни в масиві других різниць D2, потім беремо префіксні суми двічі, щоб отримати кінцевий масив.

- **Quadtrees & KD-trees / Дерева просторового розбиття: [L4]**

Симптом: 2D або 3D запити: «які точки лежать у цьому колі/прямокутнику?»

Імплементація: KD-Tree. Рекурсивне розбиття масиву точок: на парних глибинах ділимо медіаною по X , на непарних — медіаною по Y (`std::nth_element`). Пошук з відсіканням гілок.

- **3D Geometry / 3D Геометрія: [L4]**

Симптом: Площини, сфери, відстань від точки до прямої в 3D.

Імплементація: Векторний добуток у 3D повертає **вектор**, ортогональний до обох вхідних векторів: $(Y_1 Z_2 - Z_1 Y_2, Z_1 X_2 - X_1 Z_2, X_1 Y_2 - Y_1 X_2)$. Використовується для знаходження нормалі до площини. Відстань від точки до площини обчислюється через скалярний добуток.

- **Slow Delaunay triangulation / Тріангуляція Делоне (Повільна): [L4]**

Симптом: З'єднати точки так, щоб у жодному описаному колі трикутника не було інших точок (максимізація мінімального кута). Або побудувати діаграму Вороного.

Імплементація: $O(N^4)$ — перебираємо всі трійки точок (трикутники) і перевіряємо, чи є в їхньому описаному колі хоч одна четверта точка. $O(N \log N)$ версія (Divide & Conquer) настільки велика, що в умовах олімпіади III її майже гарантовано напише з багами. Просіть $O(N^3)$ алгоритм через побудову опуклої оболонки у 3D (спроектувавши точки на параболоїд $Z = X^2 + Y^2$).

Порада для промптів (Геометрія): Геометрія боїться поділів. Завжди нагадуйте III: "Avoid floating-point operations (double) and division if possible. Use cross products with long long to compare fractions (e.g., instead of $A/B < C/D$, check $A \cdot D < C \cdot B$). Avoid atan2 for sorting."

РОЗДІЛ 8. АЛГОРИТМИ ПОШУКУ ТА ОПТИМІЗАЦІЯ

ГІД ПО ВИБОРУ (HUMAN LOGIC)

- «Знайти максимальне X , при якому умова ще виконується» → **Binary Search on Answer** (Бінарний пошук по відповіді).
- «Знайти точку максимуму/мінімуму опуклої або увігнутої функції» → **Ternary Search** (Тернарний пошук).
- «Експоненціальний перебір $O(2^N)$, але $N = 40$ » → **Meet-in-the-middle** (Зустріч на середині). Зменшує складність до $O(2^{N/2})$.
- «Знайти найкоротший шлях на величезній неявній сітці/графі (напр. П'ятнашки)» → **Best-first (A*)** або **Bidirectional search** (Двонаправлений пошук).

АЛГОРИТМИ ПОШУКУ (SEARCH ALGORITHMS)

- **Binary Search / Бінарний пошук:**

Симптом: Пошук по монотонній функції (умова типу 00001111 або 11110000).

Імплементація (КРИТИЧНО!): Щоб уникнути заиклень, просить ІП писати цикл `while (R - L > 1)`. Відповідь завжди буде в L або R . Для дробових чисел (double) — цикл на 100 ітерацій (`for (int i = 0; i < 100; ++i)`).

- **Ternary Search / Тернарний пошук:**

Симптом: Знайти екстремум unimodal (унімодальної) функції $f(x)$.

Імплементація: Розбити відрізок на 3 частини: $m_1 = L + (R - L)/3$, $m_2 = R - (R - L)/3$. Порівняти $f(m_1)$ та $f(m_2)$ і відкинути третину відрізка. Також можна робити **Binary search on derivative** (Бінарний пошук по похідній), якщо її легко обчислити.

- **Meet in the middle / Зустріч на середині: [L4]**

Симптом: Треба перебрати 2^{40} підмножин (рюкзак або точна сума).

Імплементація: Розділити масив на дві половини по 20. Згенерувати всі 2^{20} сум для кожної половини, відсортувати другу половину і для кожної суми S_1 з першої шукати ($Target - S_1$) у другій через `std::lower_bound`.

- **A* Search & Bidirectional / Евристичний пошук: [L4]**

Симптом: Знайти шлях між станами. Стан надто складний для BFS.

Імплементація: A* використовує пріоритетну чергу, де вага = (пройдена відстань) + (евристична оцінка до фінішу). Евристика має бути "допустимою" (ніколи не переоцінювати відстань). Двонаправлений BFS запускає "хвилю" з початку і з кінця одночасно, що зменшує час з $O(B^D)$ до $O(B^{D/2})$.

- **Iterative deepening DFS / IDA*: [L4]**

Імплементація: Запуск DFS з обмеженням глибини. Якщо не знайшли — збільшуємо ліміт. Потребує пам'яті $O(Depth)$, що рятує від Memory Limit у звичайного BFS/A*.

РОЗДІЛ 9. ТЕОРІЯ ІГОР (GAME THEORY)

ОСНОВНИЙ СТЕК (GAMES & NIM)

- **Combinatorial games, Minimax & Alpha-beta pruning:**

Симптом: Два гравці, кінцеве дерево гри, нульова сума. Наприклад, Хрестики-Нулики.

Імплементація: Рекурсивна функція. Alpha-beta pruning дозволяє відсікати гілки, які вже гарантовано гірші за знайдений оптимум.

- **Nim / Гра Нім:**

Симптом: N купок камінців. Гравці по черзі беруть будь-яку кількість камінців з однієї купки. Програє той, хто не може зробити хід.

Імплементація: Теорема Бутона. Знайти XOR-суму всіх розмірів купок: $X = A_1 \oplus A_2 \oplus \dots \oplus A_N$. Якщо $X = 0$, другий гравець виграє. Якщо $X \neq 0$, виграє перший (і його хід — взяти камінці так, щоб XOR став рівним 0).

- **Grundy numbers (Nim-values) / Числа Гранді: [L4]**

Симптом: Гра розпадається на декілька незалежних ігор. Ходити можна в будь-якій з них.

Імплементація: Будь-яка справедлива гра еквівалентна купі в грі Нім. Число Гранді стану $G(u) = MEX(\{G(v_1), G(v_2), \dots\})$. Відповідь: XOR-сума чисел Гранді всіх незалежних ігор.

- **Games on graphs / Ігри на графах (з циклами): [L4]**

Симптом: Рух фішки по графу. Хто не може зробити хід — програв.

Імплементація: Ретроспективний аналіз. Вершина виграшна (WIN), якщо з неї є хоч один хід у LOSE. Вершина програшна (LOSE), якщо ВСІ ходи з неї ведуть у WIN. Якщо є цикли, залишаються непозначені вершини (НІЧІЯ / DRAW).

РОЗДІЛ 10. БАЗОВІ ПАРАДИГМИ ТА СОРТУВАННЯ

ГІД ПО ВИБОРУ (HUMAN LOGIC)

- «Знайти відрізок масиву з максимальною сумою» → **Kadane's Algorithm** (Max contiguous subvector sum).
- «Побудувати оптимальний код / стиснути дані / злити масиви найдешевше» → **Huffman encoding** (через 'priority_queue').
- «Виконувати задачі за розкладом, щоб максимізувати прибуток» → **Greedy (Scheduling)**. Сортують за правильними критеріями.
- «Швидко відсортувати 10^7 цілих чисел» → $O(N \log N)$ повільно. Потрібен **Radix Sort** за $O(N)$.

ОСНОВНІ ПАРАДИГМИ (PARADIGMS)

- **Divide and conquer / Розділяй і володарюй:**
Симптом: Задачу можна розбити на 2 незалежні половини, розв'язати їх і злити (Merge).
Імплементація: Класика — підрахунок інверсій у масиві (Merge Sort) або пошук найближчих точок на площині.
- **Master theorem / Основна теорема про рекурентні співвідношення:**
Симптом: Треба оцінити асимптотику рекурсії виду $T(n) = aT(n/b) + O(n^d)$.
Знання: Якщо $\log_b a < d \rightarrow O(n^d)$. Якщо $\log_b a = d \rightarrow O(n^d \log n)$. Якщо $\log_b a > d \rightarrow O(n^{\log_b a})$.
- **Greedy algorithm & Scheduling / Жадібні алгоритми:**
Симптом: Локально оптимальний вибір веде до глобального оптимуму.
Імплементація: Найважче — знайти правильний компаратор для сортування. Наприклад, щоб розставити задачі з дедлайнами D_i та часом виконання T_i , сортуйте за зростанням дедлайну, а при перевищенні викидайте найдовшу задачу через 'std::priority_queue'.
- **Amortized time complexity / Амортизована складність та Інваріанти (Two Pointers):**
Симптом: Знайти кількість підмасивів, сума яких $\leq X$, або кількість унікальних чисел. Внутрішній цикл 'while' рухає правий або лівий вказівник тільки вперед.
Імплементація: Класичний метод Двох Вказівників. Навіть якщо всередині є 'while', обидва вказівники проходять масив лише один раз, даючи чисте $O(N)$.
- **Max contiguous subvector sum / Алгоритм Кадане (Kadane's):**
Симптом: Максимальна сума на підвідрізку (можуть бути від'ємні числа).
Імплементація: $O(N)$. `sum = max(A[i], sum + A[i]); global_max = max(global_max, sum)`.

АЛГОРИТМИ СОРТУВАННЯ (SORTING)

- **QuickSort / Сортують Хоара ($O(N \log N)$):** Стандартний `std::sort`. Працює in-place.
- **MergeSort / Сортують злиттям ($O(N \log N)$):** Стабільне сортування (`std::stable_sort`). Використовується для підрахунку інверсій.
- **HeapSort / Пірамідаліне ($O(N \log N)$):** Найгірший випадок строго $\log N$, не потребує додаткової пам'яті.
- **Counting Sort / Сортують підрахунком ($O(N + K)$):** Використовувати, якщо всі числа в межах від 0 до 10^6 . Просто рахуємо частоту чисел у масиві `cnt[X]++`.
- **Radix Sort / Порозрядне сортування ($O(N \cdot D)$): [L4]**
Симптом: Треба відсортувати 10^7 чисел, і $N \log N$ ловить TL.
Імплементація: Розбити числа на байти (base-256) і виконати Counting Sort по кожному байту 4 рази (для 32-бітних чисел). Це працює блискавично швидко на реальному "залізі".
- **Finding interesting points in $N \log N$:** Сортують подій (Sweep Line) або перетин інтервалів.

PROMPT-TRD ОФІЦІЙНО ЗАВЕРШЕНО!

Загальна стратегія на контекст: 1) Визначте симптом задачі по N . 2) Знайдіть її у Гіді. 3) Скопіюйте "Target complexity" та англійський термін алгоритму з імплементацією у промпт до LLM. 4) Submit -> Accepted!